MAWE — Multi-Agent AI Workflow Engine

Simulating an Autonomous Software Development Team

Capstone project · Department of Computer Science · Southwest Minnesota State University

Marco Perozo · Gabriel Marte · Lewdy Aquino · Alvin Almonte

Everything for the presentation, in one file.

#	Document
1	Abstract
2	Presentation script
3	Demo runbook
4	How it works, and how to run it
5	Final report
6	Conference poster
7	Slide deck

One thing to state up front. The engine, the benchmark and the comparison are built, tested and validated. Two of the five hypotheses are answered by measurement. The other three compare language models and have **not** been run: that needs a paid API key. The apparatus for it is complete.

1. Abstract

MAWE — Conference abstract

Multi-Agent AI Workflow Engine: Simulating an Autonomous Software Development Team

Marco Perozo, Gabriel Marte, Lewdy Aquino, Alvin Almonte
Department of Computer Science, Southwest Minnesota State University

> Generated from `findings.json`. Every figure matches what the code currently does.

What an abstract is

One self-contained paragraph, normally 150–250 words, that makes sense to someone who has not seen the poster. It answers five questions in order:

1. **What problem?**
2. **What did you build?**
3. **How did you evaluate it?**
4. **What did you find?**
5. **Why does it matter?**

It is not an introduction, and it does not describe the document. “This poster presents our approach and discusses results” says nothing — the results themselves say everything. Write it last, in the past tense, with no citations and no undefined abbreviations.

Full version — 244 words

for a submission form that allows 250 words

Most AI coding assistants are a single model answering a single prompt: one pass to understand, design, write and verify. We present MAWE, a Multi-Agent AI Workflow Engine that divides the work across three agents mirroring a software team — a Product Manager that turns a request into a specification with executable acceptance checks, a Developer that implements it, and a Code Reviewer that runs the code in a sandbox and either approves it or sends it back. The agents are orchestrated as an explicit state graph whose revision loop is bounded by five exit conditions, so it cannot run forever. We evaluate MAWE against three controls — a single model, a single model that critiques and revises its own code, and an ablation with context compression disabled — on a benchmark of 20 Python and JavaScript tasks carrying 181 hidden tests no agent sees, so scoring does not depend on the reviewing model. The benchmark is itself validated: every task is solvable, none is satisfied by a program that does nothing, and injected defects are detected 91% of the time. Two results are established by measurement. Across 100,000 randomised adversarial review sequences, no run failed to terminate and none exceeded its iteration cap; and compressing the revision history cuts prompt size by 94% at 80 iterations while leaving outcomes unchanged. Whether role separation improves correctness, and whether it justifies its token cost, requires a comparison against a live model that has not yet been run.

Medium version — 140 words

for a 150-word limit

Most AI coding assistants are a single model answering a single prompt. We present MAWE, a Multi-Agent AI Workflow Engine that divides the work across three agents mirroring a software team: a Product Manager that writes a specification with executable acceptance checks, a Developer that implements it, and a Code Reviewer that runs the code in a sandbox and sends it back until it passes. We evaluate it against a single model, a self-refining single model, and an ablation, on 20 tasks with 181 hidden tests no agent sees, so scoring does not depend on the reviewing model. Across 100,000 randomised adversarial sequences, every run terminated inside its iteration cap, and compressing the revision history cut prompt size by 94% without changing outcomes. Whether role separation improves correctness, and at what cost, awaits a comparison against a live model.

Elevator version — 69 words

for a programme listing, or for answering "so what is it?" out loud

MAWE divides AI code generation across three agents — a Product Manager, a Developer and a Code Reviewer that runs the code — instead of asking one model to do everything in one pass. On a 20-task benchmark with 181 hidden tests, we show the revision loop always terminates (100,000 adversarial sequences, none exceeding its cap) and that compressing the revision history cuts prompt size by 94% without changing outcomes.

If you have to cut it further

Cut in this order, and stop as soon as it fits:

1. the benchmark-validation sentence (solvable / non-vacuous / 91% detection);
2. the names of the three control conditions, keeping "three controls";
3. the description of what each agent does, keeping the three role names.

Do **not** cut the last sentence. An abstract that omits the fact that no live model has been run would be claiming more than the work supports, and that is the one error an examiner is certain to catch.

2. Presentation script

MAWE — final presentation script

Marco Perozo · the finished project · Phase 4 complete

Target: **8 minutes** of talking with the demo running inside it, then questions. Cut sections 4 and 6 if you only have five.

Have open before you start 1. A terminal in the project folder, large font, nothing else on screen. 2. `docs/reports/FINAL-REPORT.pdf`. 3. GitHub, on the Actions tab, so the green ticks are one click away.

Run the demo with

```
powershell -ExecutionPolicy Bypass -File scripts\demo.ps1
```

or double-click `demo.bat`. It pauses before every step and prints the line to say, so you are never typing while someone watches.

> Everything in > below is meant to be said out loud. Everything in *italics* is a stage direction. Do not read the italics.

---

0 · Opening — 20 seconds

> "Our project is the Multi-Agent AI Workflow Engine — MAWE. The one-line version: you give it a plain-English idea, and a team of AI agents plans it, builds it, reviews it, and fixes it until it works.
>
> Before I show you anything, one thing up front. Everything is built and tested, and two of our five hypotheses are answered by measurement. The other three compare AI models against each other, and **we have not run that comparison** — it needs a paid API key. The machinery for it is finished and validated. I'll come back to that at the end."

Say the limitation here, in the first twenty seconds. If you raise it, it reads as rigour. If he finds it, it reads as an oversight. This single paragraph is the most important thing in the script.

---

1 · The problem — 45 seconds

> "A normal AI coding assistant is one model answering one prompt. In a single pass, that model has to understand what you meant, design a solution, write the code, and check its own work. It usually produces something plausible — and it has nobody to catch what it missed.
>
> We split that into three specialists and a loop, the way a real team works. A **Product Manager** turns a vague request into a precise specification. A **Developer** writes the code against it. A **Code Reviewer** checks it, and if it fails, it goes back to the Developer with the

specific problems.

The research question is not 'can agents write code' — they can. It's whether splitting the work up produces **more correct** software than one model doing everything, and what that costs."

---

2 · The one design decision that matters — 45 seconds

"If the Reviewer is just another model reading code and giving an opinion, you've built an expensive way to get a second guess.

So the Product Manager doesn't only write acceptance criteria in English. It writes **runnable checks** — 'run this program with the argument one hundred C, and standard output must be exactly two-one-two point zero'. The Reviewer executes those in a sandbox before it forms a verdict.

Where the model's opinion and the program's actual behaviour disagree, **the measurement wins**. A model cannot talk its way past a failing program. That is the difference between this and asking the AI twice."

---

3 · Show it working — 2 minutes

Run step 2 of the demo. While the output scrolls:

"Watch the Product Manager write five acceptance criteria. The Developer implements it. Then the Reviewer runs it — and rejects it, for a specific reason: self-collision never ends the game.

Second iteration: the Developer fixes that exact thing. The second review finds only a naming nit — severity 'minor', which doesn't block — so the run is approved.

Two iterations, and the thing that ended the loop was a measurement, not an opinion."

---

4 · The hard part — 1 minute 15 *(the strongest CS content)*

"Wiring three prompts together is easy. Two things make it hard, and they are where the actual engineering is.

First: the loop might never stop. Two agents disagreeing is an infinite loop, and every cycle costs money — an unattended run could burn the whole API budget overnight. So there are five ways out. The subtle one is oscillation: if you only check 'is this the same as last time', code that alternates between two versions passes that check forever. We detect that separately."

Run step 3 — the stress test. It takes about two seconds.

"A hundred thousand randomised adversarial review sequences. None failed to terminate.

None exceeded their iteration cap.

Second: the conversation grows forever. By iteration twenty you're re-sending the entire history every time and paying for every token. So older iterations collapse into a structured log. Deliberately **not** by asking a model to summarise — a summariser that hallucinates would corrupt the Developer's only memory of what it already tried. At eighty iterations the prompt drops from fifty-two thousand tokens to three thousand, a ninety-four percent reduction, with no change in outcome."

---

5 · How we tested it with no API key — 1 minute 30

This question is coming whether or not you invite it. Invite it.

"You might reasonably ask how a project that has never called a real AI model can claim to be tested.

The answer is a distinction. **Testing the machinery around a model and testing the quality of a model are two different jobs.** The first needs no API key. The second does, and we haven't done it.

Every agent talks to one interface with a single method. A real provider implements it by calling Anthropic; our mock implements it by replaying a scripted answer. There is no test mode anywhere — the engine never imports the mock and cannot tell the difference. Swapping to a real model is one environment variable.

And the generated code is real. It's written to disk and executed in a subprocess, and we compare the actual bytes of output. No API involved in that at all.

Some things can **only** be tested this way. To prove the no-progress exit fires, something has to return byte-identical code twice. No real model will do that reliably — they're non-deterministic by design."

If he looks sceptical, run step 4. It demonstrates all of that in twenty seconds, and ends by stating what none of it proves.

---

6 · The benchmark and the comparison — 1 minute 15

"For the comparison we built a benchmark: twenty tasks in Python and JavaScript, with a hundred and eighty-one hidden tests the agents never see. That matters — it means the score doesn't depend on our own reviewing model. A run our Reviewer approved can still score zero.

And we measured the benchmark itself rather than assuming it was any good. Every task has a reference solution that passes all its hidden tests. A program that starts and does nothing solves zero of twenty. And when we inject deliberate single-line bugs into the reference solutions, the hidden tests catch ninety-one percent of them.

That last check earned its place — it found two real bugs in our own test suite. Every error-

path test was asserting 'exit status one', which an uncaught exception also produces, so a file of prose was scoring three out of seven on FizzBuzz."

Only run step 6 if you have time. If you do, say the warning sentence before the numbers appear:

> "These numbers come from a fixture with known answers, not a real model. They show the experiment pipeline works. They say nothing about any model's ability."

---

7 · Close — 40 seconds

> "So, where it stands. Four milestones, all met. The engine works, the benchmark is validated, the comparison apparatus is built and proven to detect a difference.
>
> Two hypotheses answered by measurement: the loop always terminates, and the memory compression works without changing outcomes.
>
> Three not answered: whether role separation actually produces more correct software, whether it covers requirements better, and whether it's worth the cost. Those need one sweep against a real model — two hundred and forty runs, about an hour of compute.
>
> What's missing is an API key, not code."

Then stop talking.

The three questions most likely to come back

"Why didn't you just get an API key?"

> "Budget — the proposal allowed fifty to a hundred and fifty dollars for the semester and we prioritised building something worth spending it on. The sweep is one command and the cost estimate prints before it runs."

"Isn't the mock just avoiding the hard part?"

> "It's the opposite — some guarantees can only be tested that way. No real model will return byte-identical code on demand, or refuse to approve anything fifty times in a row. Those adversarial cases are exactly what a safety guarantee depends on."

"What's the weakest part of this?"

> "Two things. The live model run, which I mentioned. And our comparison between the full engine and the self-refining baseline changes two variables at once — role separation and the sandbox-grounded review — so a difference couldn't be attributed to one of them. We report that rather than hide it. Separating them would need a fifth condition."

Answering that last one well is worth more than any part of the demo. It shows you understand what your own experiment can and cannot support, which is the thing being assessed.

If something breaks live

Don't debug in front of him. Say:

> "That's not behaving — let me show you the recorded result instead."

Then open `docs/reports/FINAL-REPORT.pdf` and the Actions tab. Every number in that report was generated from a measurement, and the green ticks prove the suite passes on three Python versions on machines that aren't this one.

3. Demo runbook

Showing MAWE to Dr. Hezy — a runbook

Everything you need in the order you need it. Follow it top to bottom.

---

0. Is the project ready to show?

Yes — with one gap you should state yourself before he finds it.

The engine	works end to end
The benchmark	built and validated
The comparison	built and validated
Tests	the full suite passes, on three Python versions
All four milestones	met
The one gap	**no run against a real AI model**

The gap is real and you should name it in your first two minutes. It is not a flaw in the work; it is the boundary of what the work currently shows, and the project is explicitly honest about it in the report, the deck and the poster.

The best way to say it:

> "Everything is built and tested, and two of our five hypotheses are answered by measurement. The other three compare AI models, and we haven't run that comparison yet because it needs a paid API key. The machinery to run it is finished — that's about an hour of compute once we have the key."

Say it early. A limitation you raise yourself reads as rigour; the same limitation found by your examiner reads as an oversight.

---

1. Five minutes before he arrives

Open a terminal in the project folder and run this once, so nothing is downloading or compiling while he watches:

```
python -m pytest -q
```

It should end with a line saying everything passed. If it does, you are ready.

Have these open in tabs:

1. the terminal, in the project folder

2. `docs/reports/FINAL-REPORT.pdf`
3. the GitHub repository, on the Actions tab

---

2. The demo, in order

Six commands. You do not need all six — **A and B alone make the case**, and E is the one to run if he asks how any of this was tested without an API key.

A. The tests — start here

```
python -m pytest -q
```

Say: "The whole project is covered by automated tests, and they run on Python 3.11, 3.12 and 3.13 in continuous integration on every push. No API key, no network, no cost."

Why this first: it is the single strongest thing you can show, and it takes about a minute.

B. Watch the agents work

```
python -m mawe.cli "Build a terminal snake game" --verbose
```

Say: "The Product Manager writes a specification with acceptance checks. The Developer implements it. The Reviewer actually *runs* the code in a sandbox and compares the output to the spec — it does not just read the code and give an opinion. If it rejects, the work goes back to the Developer."

If he asks whether that's a real AI model: "No — this is our mock provider, which replays scripted answers. It's deliberate: it makes the run deterministic and free, and it's the only way to test things a real model can't be made to do on demand, like returning byte-identical code twice."

C. The loop always terminates — the computer-science contribution

```
python scripts/week6_loop_stress.py
```

Say: "Two agents arguing is a loop, and a loop that doesn't terminate burns your entire API budget overnight. We drive a hundred thousand randomised adversarial review sequences through the routing policy. None fail to terminate and none exceed their iteration cap."

This is the result to lead with if he asks what is *novel*.

D. The benchmark is a valid instrument

```
python scripts/week9_bench_check.py
```

Takes about two minutes. Add `--quick` to skip the slow part.

Say: "Twenty tasks with hidden tests the agents never see, so scoring doesn't depend on our reviewing model — a run our own Reviewer approved can still score zero. And we measure the benchmark itself:

every task is solvable, no task can be passed by a program that does nothing, and when we inject deliberate bugs into the reference solutions the hidden tests catch 91% of them."

Why this matters: most student projects assert their benchmark is good. This one measures it.

E. How any of this is tested with no API key

Keep this one in your back pocket. Run it the moment he asks.

```
python scripts/how_we_test.py
```

It demonstrates, rather than claims: the single-method interface the engine talks to, a real program executed in a subprocess with an off-by-one error caught by running it, a termination guarantee that *only* a scripted provider can test, and a closing section stating what none of it proves.

F. The full comparison

```
python scripts/week11_experiment.py --tasks fizzbuzz palindrome json-query
```

Say — and do not skip this sentence: "These numbers come from a fixture with known answers, not from a real model. This shows the experiment pipeline works end to end and that the comparison can detect a quality difference. It says nothing about any model's ability."

The script prints that warning itself, so you are not relying on memory.

---

3. What to hand him

File	What it is
`docs/reports/FINAL-REPORT.pdf`	the full written report
`docs/reports/ABSTRACT.md`	the conference abstract, three lengths
`docs/poster/MAWE-URC-Poster-with-results.pdf`	the URC poster
the GitHub repository link	the code, the history and the CI record

---

4. Questions he is likely to ask

"Did you run this against GPT or Claude?"

> "Not yet. That's the one thing left. The adapters are written but have never run against a live API, so I'd call them untested rather than working. The benchmark, the baselines and the statistics are all finished and validated — it's one sweep of 240 runs once we have a key."

"So what have you actually proven?"

> "Two things, both by measurement rather than argument. The loop always terminates — a hundred thousand adversarial sequences, none failed. And compressing the revision history cuts the prompt by 94% at 80 iterations without changing the outcome. Those two don't need a model. The three that compare correctness and cost do."

"How did you test it worked, and how is that possible with no API key?"

The question most likely to decide how this goes. Answer with the distinction, then run the script.

> "Testing the machinery around a model and testing the quality of a model are two different jobs. The first needs no API key. The second does, and we haven't done it.
>
> Every agent talks to one interface with a single method. A real provider implements it by calling Anthropic; our mock implements it by replaying a scripted answer. The engine has no test mode and never imports the mock, so it genuinely cannot tell the difference — swapping to a real model is one environment variable.
>
> And the generated code is real. It's written to disk and executed in a subprocess, and we compare the actual bytes of its output against what the specification demanded. That part involves no API at all."

Then show him:

```
python scripts/how_we_test.py
```

Twenty seconds, four parts: the interface, a real program being run with an off-by-one error caught by execution, a guarantee only a scripted provider can test, and a plain statement of what none of it proves.

"Isn't using a mock avoiding the hard part?"

> "It's the opposite — some things can *only* be tested this way. Proving the no-progress exit fires needs the Developer to return byte-identical code twice. No real model will do that reliably; they're non-deterministic by design. Same for the reviewer that never approves, and code that cycles A to B and back. Those are the adversarial cases a safety guarantee depends on, and a real model won't produce them on demand."

"How do you know your benchmark is any good?"

> "We measured it. Every task has a reference solution that passes all its hidden tests. A program that starts and does nothing solves zero of twenty. And we inject single defects into the reference solutions — the tests catch 91% of them. That last one found two real bugs in our own test suite."

"What's the weakest part?"

> "Two things. No live model run, which I've mentioned. And our comparison between the multi-agent engine and the self-refining baseline changes two variables at once — role separation and the sandbox-grounded review — so a difference couldn't be attributed to one

> of them. We report that rather than hide it; separating them needs a fifth condition."

Answering that last one well is worth more than any demo. It shows you understand what your own experiment can and cannot support.

"Is the sandbox secure?"

> "No, and we say so. It's a subprocess with a timeout, path containment and output truncation — a resource limit, not a security boundary. Docker with no network was the stated hardening target and we didn't implement it."

---

5. If something breaks

Problem	Fix
`pytest` not found	Use `python -m pytest`. The `pytest.exe` shim lands in a Scripts folder that is usually not on PATH
A JavaScript task is skipped	Node isn't installed. Harmless — the 12 Python tasks still run
A script seems to hang	The benchmark runs real subprocesses; week 9 takes ~2 min. Use `--quick`
Something fails on the day	Show `docs/reports/FINAL-REPORT.pdf` and the green CI history instead. The measurements are recorded there and the Actions tab proves they ran

---

6. The one-sentence version

If you only get thirty seconds:

> "We built a system where three AI agents — a planner, a developer and a reviewer that actually runs the code — work together instead of one model doing everything in one pass. We proved the loop always terminates and that our memory compression cuts prompt size by 94% without changing results. What's left is running it against a real model to see whether the extra structure is worth the cost."

4. How it works, and how to run it

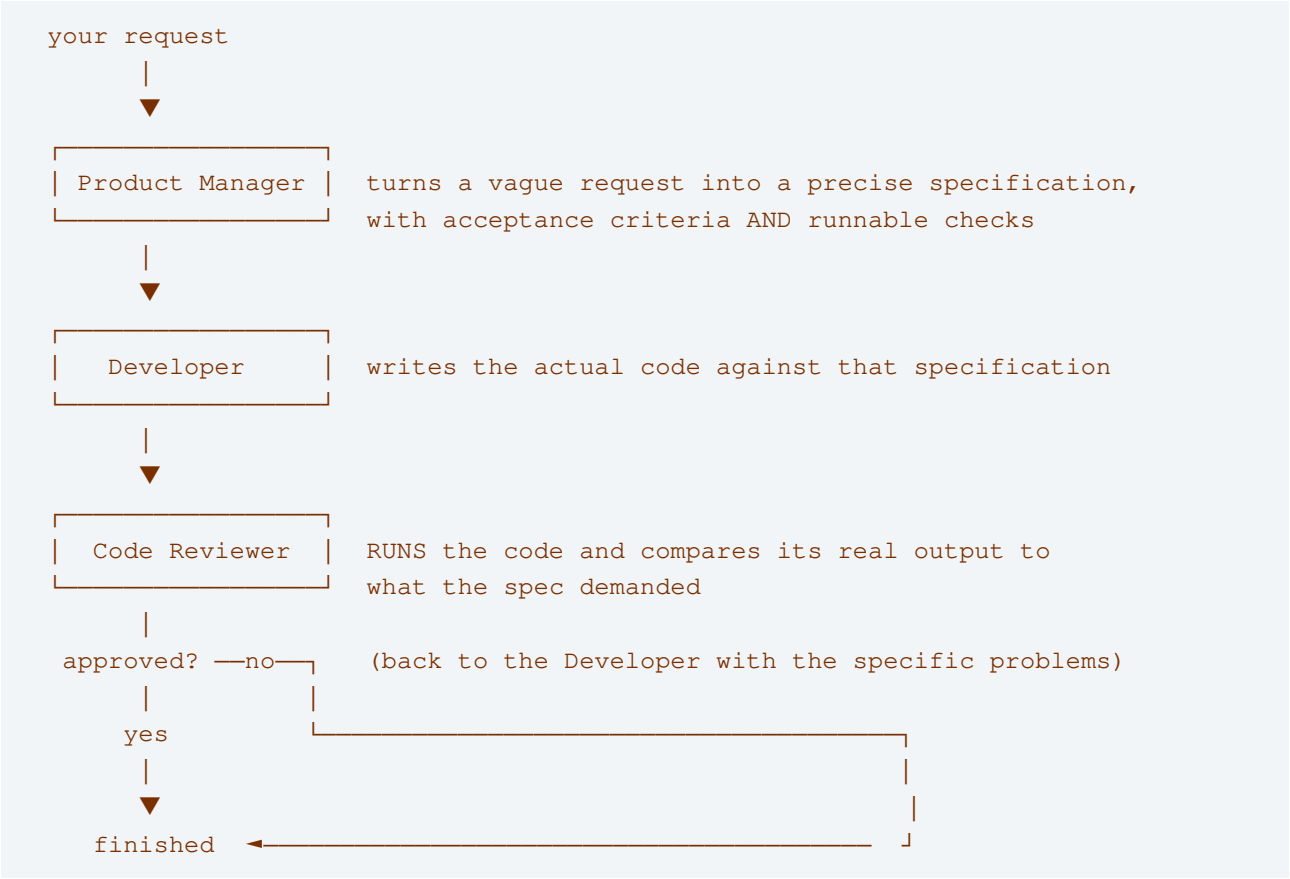
MAWE — how it works, and how to run it

Written so you can explain this project without reading from a script, and run it from a fresh machine without guessing.

Part 1 — The idea

A normal AI coding assistant is **one model answering one prompt**. You ask for a snake game, and in a single pass that model has to understand what you meant, design a solution, write the code, and check its own work. It usually gets something plausible. It also has nobody to catch what it missed, and no reason to look twice.

MAWE splits that into **three specialists and a loop**, the way a real team works:



The single most important design decision is in the Reviewer. **It does not read the code and give an opinion. It runs the code.** The Product Manager emits checks like "run this with the argument `100 C` and standard output must be exactly `212.0`", and the Reviewer executes them in a sandbox. Where the model's opinion and the program's actual behaviour disagree, **the measurement wins.** A model cannot talk its way past a failing program.

That is what makes this different from "ask the AI twice".

Part 2 — The two problems that make it hard

Anyone can wire three prompts together. The reason this is a capstone and not a weekend project is that doing so creates two problems that will destroy you if you ignore them.

Problem 1: the loop might never stop

Two agents disagreeing is an infinite loop. The Developer changes something, the Reviewer finds something else, forever — and every cycle costs money. An unattended run could burn an entire API budget overnight.

So the loop has **five ways to stop**, and a run always exits through exactly one of them:

Exit	Fires when
`approved`	the review passed
`iteration_cap`	the configured maximum was reached (default 5)
`no_progress`	the Developer returned byte-identical code twice — it isn't responding to feedback
`oscillation`	the code went back to a version already seen (A → B → A)
`repeated_issues`	the Reviewer keeps refiling the same complaint
`error`	an agent failed irrecoverably

`oscillation` is the subtle one. If you only check "is this the same as last time?", a loop that alternates between two versions passes that check forever.

How we know it works: 100,000 randomised adversarial review sequences. None failed to terminate, none exceeded its cap.

Problem 2: the conversation grows forever

Every iteration adds the previous code and the previous review to the next prompt. By iteration 20 you are re-sending the entire history every time, and you pay for every token of it.

So older iterations collapse into a structured progress log — what changed, what the review said, which checks passed — while the most recent version is kept in full.

Crucially, this is not done by asking a model to summarise. A summariser that hallucinates would corrupt the Developer's only memory of what it already tried. The fields matter, so they are recorded exactly.

How we know it works: at 80 iterations the prompt falls from 52,494 tokens to 3,316 — a 93.7% reduction — and the outcome does not change.

Part 3 — Setting it up, from nothing

What you need

- **Python 3.11 or newer** — check with `python --version`
- **Git**
- *Optional:* **Node.js**, only for the 8 JavaScript benchmark tasks. Without it they are skipped and everything else still works.
- **No API key.** Nothing in this guide costs money.

Where to run it

Open a terminal — on Windows, PowerShell or the terminal inside VS Code — and go to the project folder. On this machine that is:

```
cd C:\Users\marco\Documents\MAWE-Capstone
```

If you are on a different machine, clone it first:

```
git clone https://github.com/marcoaperozo/Capstone-MAWE.git
```

```
cd Capstone-MAWE
```

Install, once

```
pip install -r requirements.txt
```

```
pip install -e .
```

The second line installs the project itself, so `python -m mawe.cli` works from anywhere in the folder. You only do this once per machine.

Check it worked

```
python -m pytest -q
```

It should finish in about a minute with a line saying everything passed, and some tests **skipped**. Skips here are expected and correct: they are the statistics tests that check our maths against SciPy, which is not a required dependency. Install SciPy and they run too. Nothing is skipped because it is broken.

Deliberately not printed here: the exact number of tests. It changes as the project grows, and a number written into a guide goes stale — this project had to correct exactly that mistake four times. The figure that *is* kept current lives in `docs/reports/FINAL-REPORT.md`, which is generated from measurements.

Part 4 — Running it

Watch the agents work

```
python -m mawe.cli "Build a terminal snake game" --verbose
```

You will see this:

```
  [plan]      specification written, 5 acceptance criteria
              1. the program runs without raising an exception
              2. the snake grows by exactly one segment per food eaten
              3. colliding with a wall ends the game
              ...
  [develop]   iteration 1: src/main.py
  [review]    FAIL  severity=major  tests 2 passed / 0 failed
              1. criterion 4 fails: self-collision never ends the game
  [develop]   iteration 2: src/main.py
  [review]    FAIL  severity=minor  tests 2 passed / 0 failed
              1. all 5 criteria pass; the name 'grow' shadows a local
  [finalize]  approved
```

Read that carefully, because it is the whole project in one screen. The Reviewer rejected the first attempt for a specific, concrete reason. The Developer fixed that exact thing. The second review found only a style nit — severity `minor`, which does not block — so the run was approved.

Useful options

Option	What it does
`--verbose`	show each step as it happens
`--out ./generated`	write the finished code to a folder so you can run it
`--db sqlite:///demo.db`	use a specific database file
`--no-sandbox`	skip executing the code (faster, less evidence)
`--max-iterations 3`	change the loop cap

To actually keep and run the generated program:

```
python -m mawe.cli "Build a command-line calculator" --out ./generated
```

Part 5 — Showing progress

Four independent records, and they agree with each other.

1. The git history — the week-by-week story

```
git log --oneline
```

One commit per week, each carrying that week's result in its message. This is the clearest evidence that the project was built steadily rather than the night before.

2. The database — every run is traceable

Every run records what happened, in 11 tables. After running the CLI with `--db sqlite:///demo.db`:

```
python -c "import sqlite3; c=sqlite3.connect('demo.db'); [print(r) for r in c.execute(
```

```
(0, 'product_manager', '{\n  "summary": "Terminal snake game on a 20x20 grid.",\n  "la
(1, 'developer', 'produced src/main.py')
(1, 'reviewer', 'criterion 4 fails: self-collision never ends the game')
(2, 'developer', 'produced src/main.py')
(2, 'reviewer', "all 5 criteria pass; the name 'grow' shadows a local")
```

Read the first column. The Product Manager spoke once, before any iteration. Then Developer and Reviewer alternate, and **each review is filed against the iteration it actually judged** — which is worth pointing out, because it was wrong until recently: a stale database relationship meant every review after the first was recorded against iteration 1. The run finished correctly, so only the record was wrong, and the record is what we offer as proof a run is traceable.

Every version of the code is kept with a SHA-256 hash — that hash is what the no-progress detector compares.

3. Continuous integration — it works on machines that are not yours

Every push runs the full suite on Python 3.11, 3.12 and 3.13 on GitHub's servers. The green ticks at the Actions tab are independent proof: nothing is passing because of something peculiar to your laptop.

4. The measured results

Command	What it proves	Time
`python -m pytest -q`	everything works	~1 min
`python scripts/how_we_test.py`	how it is tested with no API key	20 s
`python scripts/week6_loop_stress.py`	the loop always terminates	~2 s
`python scripts/week7_memory_check.py`	the context stays bounded	~5 s
`python scripts/week9_bench_check.py`	the benchmark is valid	~2 min
`python scripts/week11_experiment.py`	the four-condition comparison	~3 min
`python scripts/week12_final_check.py`	the final gate, all of it	~2 min

That last one rewrites `docs/reports/findings.json`, which the final report and the deck are

generated from. **No figure in either document is typed by hand** — rerun it and both can be rebuilt.

Part 6 — The honest boundary

Everything above runs against a **mock provider**: a stand-in that replays scripted answers instead of calling a real AI model.

That is a deliberate choice, not a shortcut, and the distinction to hold on to is this: **testing the machinery around a model and testing the quality of a model are two different jobs.** Routing, loop prevention, sandboxed execution, scoring, persistence and token accounting are all machinery, and the mock tests them perfectly. Some of them can *only* be tested this way — proving the no-progress exit fires requires something to return byte-identical code twice, and no real model will do that reliably.

What it does **not** show is whether a real model writes good code. That needs an API key, and it has not been run. The benchmark for it is built and validated — 20 tasks, 181 hidden tests, four conditions, 240 runs — so what is missing is the key, not the code.

Run `python scripts/how_we_test.py` to see that argument demonstrated rather than asserted.

If something goes wrong

Problem	Fix
`python` not recognised	Python isn't installed or isn't on PATH. Reinstall and tick "Add Python to PATH"
`No module named mawe`	You skipped `pip install -e .`, or you are in the wrong folder
`pytest` not recognised	Use `python -m pytest` instead. pip installs `pytest.exe` into a Scripts folder that is often not on PATH; the module form always works
JavaScript tasks skipped	Node isn't installed. Harmless — the 12 Python tasks still run
A script seems stuck	The benchmark runs real programs; week 9 takes ~2 minutes. Add `--quick`
27 tests skipped	Correct. Those compare our statistics against SciPy, which is optional

5. Final report

MAWE — Multi-Agent AI Workflow Engine

Final report

Capstone project · Department of Computer Science · Southwest Minnesota State University

Marco Perozo · Gabriel Marte · Lewdy Aquino · Alvin Almonte

Engine v1.0.0 · 445 automated tests · figures generated 2026-11-02

> Every figure in this report is generated from `docs/reports/findings.json`, which `scripts/week12_final_check.py` writes from live measurements. Nothing here is typed in by hand. Re-run that script and this document can be rebuilt.

---

1. What this is, and what it found

Most AI coding tools are one model and one prompt: the model has to understand the request, design a solution, write it and check it in a single pass. MAWE splits that across three agents — Product Manager, Developer, Code Reviewer — on a LangGraph state graph, and loops until the code passes review.

The research question is not whether agents can write code. It is whether role separation produces **more correct, more requirement-compliant software than a single model**, and what that costs.

The honest summary at the end of twelve weeks:

- The **engine works** and is tested: 445 automated tests across 14 modules, green on Python 3.11, 3.12 and 3.13.
- The two hypotheses that **do not need a language model** — termination and context compression — are **answered by measurement**.
- The three that **do** need one — correctness, specification coverage, cost per success — are **not yet answered**. The instrument to answer them is built and validated; no sweep against a real model has been run.

That last point is the main limitation of this report, and it is stated here rather than buried in section 7.

2. The experimental design

Four conditions, run over the same tasks under the same settings:

	condition	what it is
A	single model	one call, no iteration, no review
B	self-refinement	one model that critiques and revises its own code
C	multi-agent engine	Product Manager → Developer → Reviewer, looping

C'	ablation	the engine with context summarisation switched off

B is the control that matters. Without it, "multi-agent beats one call" is indistinguishable from "more attempts beat one attempt". B iterates under the same cap with the same model, so an advantage C shows over B is not simply persistence.

Everything outside the conditions is shared deliberately: the same provider instance, temperature and iteration cap, the same file parser, the same token accounting at the single call site, and the same scoring. The baselines get none of the engine's machinery — no specification, no executable checks, no sandbox evidence, no loop prevention — which is why B runs to its cap against a model that never settles.

One confound is not removed, and should not be read past. C differs from B in two ways at once: role separation *and* sandbox-grounded review. A gain of C over B cannot be attributed to either alone by this design. Giving B the sandbox would stop it isolating iteration, which is the job it is there to do, so the confound is reported rather than engineered away. Separating the two would need a fifth condition.

3. The benchmark, and why it can be trusted

20 tasks — 7 easy, 8 medium, 5 hard — across Python (12) and JavaScript (8), with 181 hidden tests that no agent ever sees. Scoring is therefore independent of the LLM reviewer: a run the Reviewer approved can still score zero, which is the project's answer to its own stated risk that the reviewer is fallible.

A benchmark is a measuring instrument, so three properties were measured rather than asserted:

property	result
solvable — a reference solution per task passes every hidden test	20/20 tasks, 181 tests
non-vacuous — a program that starts and does nothing	solves 0/20 tasks (5/181 individual tests, the empty-input cases)
discriminating — injected single defects are caught	mutation score 91.4%

Mutation testing earned its place by finding two real defects in the suite:

1. **Every error-path test asserted "exit status 1"** — which an uncaught exception also produces. Deleting an input guard left the tests green, and a file of prose scored 3 of 7 on FizzBuzz off the error cases alone. The harness now rejects a traceback or a syntax error where the task asked for a message on stderr.
2. **Four tasks had untested contract boundaries** — an index exactly one past the end, a zero-length interval, a width of 1, a cheap route discovered after an expensive one.

Surviving mutants are reported rather than chased to zero. Several are equivalent mutants with no observable effect, and closing them would mean writing tests against the reference implementation's internals — which would overfit the benchmark to one way of solving each task.

4. Results

> **These numbers come from a fixture, not a model.** The committed sweep (`fixture-sweep.json`, 3 repetitions per task) is driven by `bench.oracle`, whose answers are known in advance: the reference solution, a verified defect, or a defect it fixes on retry. It shows that the experiment pipeline works end to end and that the comparison detects a difference it was built to detect. It says **nothing** about any model's ability.

condition	tasks solved	success rate (95% CI)	mean pass rate	mean tokens
A single model	0/20	0.0% [0.0%, 16.1%]	57.4%	575
B self-refinement	20/20	100.0% [83.9%, 100.0%]	100.0%	2,548
C multi-agent engine	20/20	100.0% [83.9%, 100.0%]	100.0%	4,093
C' engine, summarisation off	20/20	100.0% [83.9%, 100.0%]	100.0%	4,093

The success-rate interval is a Wilson score interval, not the textbook normal one, which collapses to zero width at 20 of 20 and would report a 100% success rate with no uncertainty at all from twenty observations.

Paired comparisons

comparison	n	p	p (Holm)	effect
C vs A	20	0.0000	0.0000 *	+1.00 large
C vs B	0	1.0000	1.0000	+0.00 negligible
B vs A	20	0.0000	0.0000 *	+1.00 large
C vs C'	0	1.0000	1.0000	+0.00 negligible

* significant at 0.05 after Holm correction across the four comparisons.

Two methodological points, both of which would otherwise have produced a confident number rather than an error:

- **The unit of analysis is the task, not the run.** 20 tasks with 3 repetitions gives 60 rows per condition, and testing over all of them would treat three runs of one task as three independent observations — trebling *n* and shrinking every interval by about 1.7 for nothing. Repetitions are averaged within a task; they measure stability, which is reported separately.
- **The test follows the metric.** The proposal specified a paired t-test or Wilcoxon signed-rank, which is right for tokens and pass rate and wrong for the headline. Task success is binary, so nearly every paired difference is zero and the signed-rank test discards zeros: with 20 tasks and

3 disagreements it would compute a p-value from $n = 3$ while reading as a 20-task result. McNemar's exact test on the discordant pairs is the correct choice, and it makes the real sample size the headline instead of hiding it.

5. Termination (H4)

100,000 randomised adversarial review sequences across 4 policy configurations: **0 failed to terminate** and **0 exceeded a cap**.

The loop is bounded by five independent mechanisms, unified into a single policy function so that routing and reason-reporting cannot drift apart:

mechanism	fires when
iteration cap	the configured maximum is reached
no progress	two consecutive iterations produce byte-identical code
oscillation	code returns to a version already seen (A→B→A), which consecutive-repeat detection misses
repeated issues	the same complaint is refiled, compared by normalised fingerprint
error	an agent fails irrecoverably

Exit reasons under the default policy on adversarial input:

reason	share
approved	81.2%
no_progress	16.4%
error	1.6%
oscillation	0.8%
iteration_cap	0.0%
repeated_issues	0.0%

Cap exits by configuration — the detectors are what keep this low:

configuration	exits by cap
default, cap 5	0.0%
cap 1	47.0%
cap 20	0.0%

detectors off, cap 5	2.2%

The second clause of H4 is pointed the right way but not settled. Under the default policy the cap accounts for 0.0% of exits, well inside the 10% the hypothesis allows, because the other detectors catch runs first. With the detectors disabled the cap becomes the only thing stopping a run, which is what the configuration is there to show. But adversarial random input is not a language model, and the clause is a claim about real runs — so it needs a sweep against a model to confirm.

6. Context compression (H5)

Target: at least 40% reduction without changing outcomes. Measured: **93.7%** at 80 iterations, never exceeding the 4,000-token budget (largest context actually sent: 3,316).

iterations	uncompressed	compressed	reduction
1	637	637	0.0%
2	1,293	688	46.8%
5	3,260	787	75.9%
10	6,539	954	85.4%
20	13,104	1,291	90.1%
40	26,234	1,966	92.5%
80	52,494	3,316	93.7%

The compression is **structured, not generative**: older iterations collapse into a progress log built from recorded facts — what changed, what the review said, which checks passed — rather than being re-summarised by a model. A summariser that hallucinates would corrupt the Developer's only memory of what it already tried.

The ablation confirms the "without changing outcomes" half: C and C' solve the same tasks (20/20 and 20/20). On runs long enough for compression to apply at all — it needs at least two completed iterations — C' sends measurably more tokens for the same result.

7. Hypothesis scorecard

	hypothesis	verdict
H1	Role separation produces more correct software than a single model on the same tasks.	**Not yet testable**
H2	Role separation produces better specification coverage.	**Not yet testable**

H3	The improvement is worth its cost, measured per *successful* task.	**Not yet testable**
H4	Every run terminates, and fewer than 10% end by hitting the iteration cap.	**Supported**
H5	Context summarisation cuts prompt size by at least 40% without changing outcomes.	**Supported**

H1 — Not yet testable
Role separation produces more correct software than a single model on the same tasks.
Evidence: the comparison runs, and detects a quality difference it was built to detect; no sweep against a real model has been run
Caveat: needs an API key: the committed sweep is a fixture with known answers and measures the apparatus, not a model

H2 — Not yet testable
Role separation produces better specification coverage.
Evidence: the comparison runs, and detects a quality difference it was built to detect; no sweep against a real model has been run
Caveat: needs an API key: the committed sweep is a fixture with known answers and measures the apparatus, not a model; specification coverage also has no metric yet - the benchmark scores behaviour, not whether a spec was complete

H3 — Not yet testable
The improvement is worth its cost, measured per *successful* task.
Evidence: the comparison runs, and detects a quality difference it was built to detect; no sweep against a real model has been run
Caveat: needs an API key: the committed sweep is a fixture with known answers and measures the apparatus, not a model

H4 — Supported
Every run terminates, and fewer than 10% end by hitting the iteration cap.
Evidence: 100,000 randomised adversarial sequences, 0 failed to terminate, 0 exceeded a cap
Caveat: the <10% cap-exit clause is about real runs. Under the default policy the cap accounts for 0.0% of exits because the detectors catch runs earlier, which is consistent with the clause; but adversarial random input is not a model, so the clause is not settled by this

H5 — Supported
Context summarisation cuts prompt size by at least 40% without changing outcomes.
Evidence: 93.7% reduction at 80 iterations, never over budget; C and C' solve the same tasks
Caveat: the reduction is measured on the engine's own prompts, which is where it applies; 'no pass-rate drop' is confirmed against the fixture, not against a model

8. Limitations

1. **No model has been measured.** H1, H2 and H3 need a real provider. The adapters for Anthropic and OpenAI are written but have never been run against a live API, so they are untested code,

not working code.

2. **Specification coverage has no metric.** H2 needs one. The benchmark scores behaviour, not whether a specification captured the request.
3. **C vs B confounds two variables** — role separation and grounded review — as described in section 2.
4. **Twenty tasks, small to medium.** Nothing here speaks to multi-file systems, long-lived codebases, or tasks where the hard part is the design.
5. **Functional correctness only.** Not style, security, maintainability or performance.
6. **Three repetitions.** Enough to detect instability, not enough to characterise its distribution.
7. **The subprocess sandbox is not a security boundary.** It has a timeout, path containment and output truncation; Docker with no network is the hardening target and is not implemented.

9. What would finish this

In order of what each buys:

1. **One sweep against a real model.** 4 conditions x 20 tasks x 3 repetitions = 240 runs. The pipeline, the scoring and the statistics are built and validated; this is the single step between the current report and answers to H1 and H3.
2. **A specification-coverage metric**, to make H2 testable at all.
3. **A fifth condition** that gives B the sandbox, separating role separation from grounded review.

Generated from `findings.json` (2026-11-02T23:57:32) by `docs/reports/build_final_report.py`.

6. Conference poster

Multi-Agent AI Workflow Engine

Simulating an Autonomous Software Development Team

Marco Perozo • Gabriel Marte • Lewdy Aquino • Alvin Almonte

Department of Computer Science · Southwest Minnesota State University · Capstone Project, Fall 2026

Introduction & Problem Statement

Large Language Models (LLMs) can now generate source code from natural-language descriptions, and tools such as GitHub Copilot and ChatGPT are used daily by developers (Chen et al., 2021). Almost all of them are built around one model receiving one prompt: a single pass in which the model must understand the request, design the solution, write the code and verify it all at once.

Three limitations follow:

- Single-model limits. One pass over a large, multi-step task produces incomplete or incorrect results on anything non-trivial.
- Lack of teamwork. Real software needs a product manager to clarify, a developer to build and QA to verify. A single prompt collapses these perspectives and loses the checks and balances that make teams effective.
- Lack of iteration. Standard prompts do not reliably self-correct against predefined requirements (Madaan et al., 2023); detecting and fixing errors falls to the human.

RESEARCH QUESTION

Does a structured, role-based multi-agent workflow (PM → Developer → Reviewer with an iteration loop) produce more correct and more requirement-compliant software than a single LLM given the same task — and at what cost in tokens and time?

Goal & Objectives

Goal. Design, implement and evaluate an engine that simulates an autonomous software development team and turns a simple prompt into working, reviewed code through a controlled iteration loop.

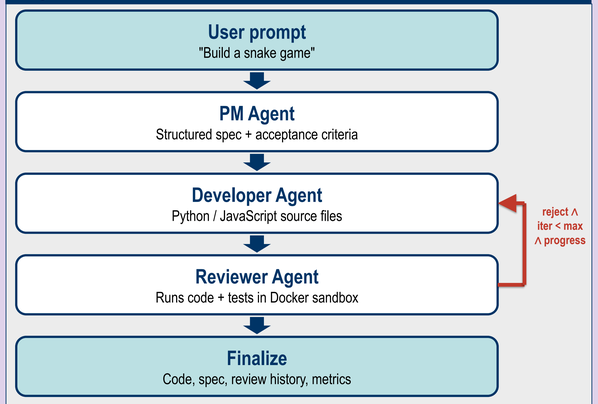
- Build the orchestration pipeline as a LangGraph state graph.
- Implement PM, Developer and Reviewer agents, each producing a well-defined artifact (spec, code, review report).
- Engineer loop prevention so the review/fix cycle always terminates.
- Implement memory management (rolling summarization and pruning).
- Persist every run, message, spec, code version, review and metric.
- Evaluate against a single-model baseline on ≥ 20 hidden-test tasks.

Related Work

Agentic frameworks show that decomposing a problem across cooperating LLMs improves quality on complex tasks: AutoGen provides a general multi-agent conversation layer (Wu et al., 2024); MetaGPT encodes standard operating procedures for software roles (Hong et al., 2024); ChatDev organises agents into a virtual software company (Qian et al., 2024). Self-Refine (Madaan et al., 2023) and Reflexion (Shinn et al., 2023) show that iterative self-feedback raises output quality, and SWE-bench (Jimenez et al., 2024) supplies a rigorous hidden-test evaluation protocol.

Gap addressed. These systems report capability, but rarely report guaranteed termination or cost per successful task. MAWE makes the control flow explicit and testable, and measures cost-per-success directly.

System Architecture



Six layers. Orchestration (LangGraph state graph) · Agent (role prompts and tools) · Safety (iteration caps, exit conditions, fallback routing) · Memory (summarization and pruning) · Persistence (SQLAlchemy / SQLite → PostgreSQL) · Evaluation (benchmarks against a single-model baseline).

Methodology

Development process. Four two-week Agile sprints; every change goes through a GitHub pull request reviewed by a teammate; GitHub Actions runs the unit-test suite on each push; 15-minute weekly stand-ups on a Kanban board.

Workflow as a state graph. Nodes are agent steps (plan, develop, review, summarize, finalize); edges are routing rules evaluated on the current state, which makes the control flow explicit, testable and easy to visualise.

START → PM.plan → Dev.implement → Reviewer.review → [approve → finalize] | [reject ∧ iter < max ∧ progress → Dev.implement] | [reject ∧ (iter = max ∨ no progress) → finalize_with_warnings] → END

Testing strategy. Unit tests for routing, loop-prevention rules, summarization and database access; integration tests over the full pipeline with a mocked LLM for determinism; end-to-end runs with real LLM calls before each milestone.

Loop Prevention & Memory Management

The two failure modes that break autonomous agent pipelines are non-termination and context growth. Both are engineered against explicitly.

Four termination mechanisms

- Hard iteration cap — configurable, default 5.
- Severity threshold — only critical and major issues cause rejection; minor issues are reported but do not block approval.
- No-progress detection — a SHA-256 hash of the Developer output exits the loop when the same code or the same issue list repeats.
- Fallback routing — on exit without approval the engine finalizes the best version with a warnings report rather than failing.

Memory strategy

Only the latest spec, latest code and latest review stay in full context; older iterations are compressed into a compact progress log stored in memory_summaries, while every raw message is still persisted in the database so nothing is lost for later analysis.

Database Design

The database is the system's memory of record: every run is fully traceable from the original prompt through each code version to the final metrics. The schema is in third normal form, written with SQLAlchemy so SQLite (development) migrates to PostgreSQL unchanged.

Table	Purpose
users / projects	Owner and the original idea prompt
agents	Role, model, system prompt, version
runs	Mode, status, iterations, tokens, cost
messages	Every agent-to-agent exchange
specifications	Versioned PM spec + acceptance criteria
code_artifacts	Each file per iteration + SHA-256 hash
reviews	Verdict, severity, tests passed/failed
memory_summaries	Compressed context, tokens before/after
benchmark_tasks	Prompt, difficulty, hidden tests
evaluation_results	Metric name and value per run

USER ——< PROJECT ——< RUN ——< { MESSAGE >—— AGENT, SPECIFICATION, CODE_ARTIFACT, REVIEW, MEMORY_SUMMARY, EVALUATION_RESULT } · BENCHMARK_TASK ——< RUN

Implementation Status

Stack. Python 3.11+ · LangGraph · Anthropic / OpenAI API SDKs · subprocess sandbox with a wall-clock timeout, path containment and output truncation · pytest · SQLAlchemy over SQLite → PostgreSQL · FastAPI · Git and GitHub Actions. No GPU is required.

Docker with no network was the stated hardening target and is not implemented. Every execution goes through the subprocess path, which is a resource limit and a containment boundary rather than a security boundary.

Phase	Weeks	Milestone	Status
1. Setup & prototyping	1–2	M1 pipeline running	Met
2. Role implementation	3–5	M2 full PM/Dev/QA loop	Met
3. Memory & state	6–8	M3 engine v1.0	Met
4. Testing & evaluation	9–12	M4 benchmark + report	Met

Experimental Evaluation

Design. The same benchmark tasks are solved under four conditions using the same underlying model and temperature, so that any difference is attributable to the workflow rather than the model.

- A — Single model (baseline): one call, one answer.
- B — Single model with self-refinement: the same model writes, then reviews and fixes its own code under the same iteration cap.
- C — MAWE (proposed): PM → Dev → Reviewer with loop prevention and memory management.
- C' — Ablation: MAWE with summarization disabled.

Benchmark. ≥ 20 Python and JavaScript tasks with hidden tests the agents never see — 7 easy, 8 medium, 5 hard. Three repetitions per task per condition (240 runs) to account for LLM non-determinism; means reported with 95% confidence intervals. The test follows the metric: Wilcoxon signed-rank for tokens and pass rate, and McNemar's exact test for task success, which is binary and would otherwise be computed from a far smaller sample than it appears to use.

Metrics. Hidden-test pass rate, task success rate, specification coverage, iterations used, total tokens, estimated cost, latency, termination reason and context reduction from summarization.

Results

All four milestones met: 445 automated tests, green on Python 3.11, 3.12 and 3.13. The two hypotheses that need no language model are answered by measurement; the three that need one are not.

Hypothesis	Result
H4 every run terminates	Supported
H5 summarisation cuts context by 40%+	Supported
H1-H3 correctness, coverage, cost	Needs a model

H4. 100,000 randomised adversarial review sequences across four policy configurations: 0 failed to terminate and 0 exceeded a cap. H5. Context at 80 iterations falls from 52,494 to 3,316 tokens, a 93.7% reduction that never exceeds the budget, with C and C' solving the same tasks.

Pipeline check (fixture, NOT a model)	Solved	Pass rate
A — Single model	0 / 20	57%
B — Self-refinement	20 / 20	100%
C — MAWE	20 / 20	100%
C' — No summarization	20 / 20	100%

A fixture with known answers, not a model: it shows the comparison detects a difference it was built to detect, and says nothing about model ability.

Discussion & Conclusions

Settled. The loop provably terminates and the context stays bounded. The benchmark is validated as an instrument: every task is solvable, no task is satisfied by a do-nothing program, and injected defects are caught 91.4% of the time.

Not settled. Whether role separation produces more correct software, and whether it is worth the cost, need a sweep against a real model. The apparatus is complete; what is missing is an API key, not code.

Limitations. Scoring uses hidden tests rather than the Reviewer's verdict, since the Reviewer is itself an LLM; tasks are small to medium; results will depend on the model family; and C differs from B in two ways at once, role separation and grounded review, so a gain over B cannot be attributed to either alone.

References

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., … Zaremba, W. (2021). Evaluating large language models trained on code. arXiv:2107.03374.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., … Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. International Conference on Learning Representations (ICLR 2024).

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? International Conference on Learning Representations (ICLR 2024).

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., … Clark, P. (2023). Self-Refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems, 36.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., … Sun, M. (2024). ChatDev: Communicative agents for software development. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 15174–15186.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., … Wang, C. (2024). AutoGen: Enabling next-generation LLM applications via multi-agent conversation. Conference on Language Modeling (COLM 2024).

Acknowledgements

We thank Dr. Hezy for supervising this capstone and for feedback on the proposal and evaluation design, and the maintainers of LangGraph, whose tooling this work builds on.

POSTER TEMPLATE BY
www.PosterPresentations.com

7. Slide deck

MAWE — Multi-Agent AI Workflow Engine

Simulating an Autonomous Software Development Team

Final report · Phase 4 complete · engine v1.0.0

Marco Perozo · Gabriel Marte · Lewdy Aquino · Alvin Almonte

Department of Computer Science · Southwest Minnesota State University

Built and tested	Answered by measurement	Still open
445 automated tests engine v1.0.0	H4 termination H5 context compression	H1, H2, H3 need a real model

Every figure in this deck is generated from measurements, not typed in.

The question, and the honest answer

Most AI coding tools are one model and one prompt. MAWE splits the work across three agents — Product Manager, Developer, Code Reviewer — and loops until the code passes review.

The question is not whether agents can write code. It is whether role separation produces more correct software than a single model, and what that costs.

	Hypothesis	Verdict
H4	Every run terminates; under 10% exit by cap	**SUPPORTED**
H5	Summarisation cuts context ≥40%, outcome unchanged	**SUPPORTED**
H1	Role separation produces more correct software	**NOT YET TESTABLE**
H2	Role separation produces better spec coverage	**NOT YET TESTABLE**
H3	The improvement is worth its cost per success	**NOT YET TESTABLE**

H1–H3 need a real API key. The instrument that answers them is built and validated; no sweep against a model has been run. That is the main limitation of this report, and it is on slide 2 rather than buried at the end.

Four conditions, because the claim is comparative

Measuring the engine alone answers nothing. It needs controls.

	Condition	What it is
A	single model	one call, no iteration, no review
B	self-refinement	one model that critiques and revises its own code
C	multi-agent engine	Product Manager → Developer → Reviewer, looping
C'	ablation	the engine with context summarisation switched off

B is the control that matters.

Without it, "multi-agent beats one call" is indistinguishable from "more attempts beat one attempt". B iterates under the same cap with the same model, so an advantage C shows over B is not just persistence.

One confound is reported, not removed.

C differs from B in two ways at once — role separation and sandbox-grounded review — so a gain over B cannot be attributed to either alone. Giving B the sandbox would stop it isolating iteration, which is its job.

The benchmark is itself measured

20 tasks — 7 easy, 8 medium, 5 hard — in Python and JavaScript, with 181 hidden tests no agent ever sees. A run the Reviewer approved can still score zero.

Property	What it rules out	Result
Solvable	a task nobody can pass — which would read as "all conditions did badly"	20/20
Non-vacuous	tests a do-nothing program satisfies	0/20 solved
Discriminating	tests that miss a real defect	91.4%

Mutation testing found two real defects in the benchmark:

1. Every error-path test asserted "exit status 1" — which an uncaught exception also produces. Deleting an input guard left the tests green, and a file of prose scored 3 of 7 on FizzBuzz off the error cases alone.

2. Four tasks had untested contract boundaries: an index one past the end, a zero-length interval, a width of 1, a cheap route found after an expensive one.

Results

These numbers come from a fixture with known answers, not a model. They show the pipeline works and that the comparison detects a difference. They say nothing about any model.

Condition	Solved	Success rate (95% CI)	Pass rate	Tokens
A single model	0/20	0.0% [0%–16%]	57.4%	575
B self-refinement	20/20	100.0% [84%–100%]	100.0%	2,548
C engine	20/20	100.0% [84%–100%]	100.0%	4,093
C' engine, no summarisation	20/20	100.0% [84%–100%]	100.0%	4,093

Comparison	n	p (Holm)	Effect
C vs A	20	0.0000 *	+1.00 large
C vs B	0	1.0000	+0.00 negligible
B vs A	20	0.0000 *	+1.00 large
C vs C'	0	1.0000	+0.00 negligible

Task success is binary, so the paired test is McNemar's exact test on the discordant pairs — not Wilcoxon, which would discard the agreeing pairs and compute from a far smaller n while reading as a 20-task result.

Holm-corrected across the four comparisons.

What is answered: termination and context

H4 — every run terminates

100,000 randomised adversarial sequences across 4 policy configurations.

0 failed to terminate. 0 exceeded a cap.

Under the default policy the cap accounts for 0.0% of exits — the other four detectors catch runs first.

Five independent exits

iteration cap · no progress (identical code twice) · oscillation (a version already seen) · repeated issues (the same complaint refiled) · error

Routing and reason-reporting are one function, so they cannot drift apart.

H5 — context stays bounded

Iterations	Naive	Compressed	Saved
2	1,293	688	47%
10	6,539	954	85%
20	13,104	1,291	90%
40	26,234	1,966	93%
80	52,494	3,316	94%

93.7% smaller at 80 iterations

Never over the 4,000-token budget; the largest context actually sent was 3,316.

The compression is structured, not generative: older iterations collapse into a log built from recorded facts. A summariser that hallucinated would corrupt the Developer's only memory of what it already tried.

What this does not show

Stated plainly, because a capstone that only lists what worked is not a report.

Limitation	Why it matters
No model has been measured	H1, H2 and H3 need a real provider. The Anthropic and OpenAI adapters are written but have never run against a live API — untested code, not working code.
Spec coverage has no metric	H2 is not just unmeasured, it is not yet measurable. The benchmark scores behaviour, not whether a specification captured the request.
C vs B confounds two variables	Role separation and grounded review change together.
20 small-to-medium tasks	Nothing here speaks to multi-file systems or tasks where the hard part is the design.
Functional correctness only	Not style, security, maintainability or performance.
The sandbox is not a security boundary	Timeout, path containment and output truncation only. Docker with no network is the hardening target and is not implemented.

What would finish it

	Step	What it buys
1	One sweep against a real model (240 runs)	Answers H1 and H3. The pipeline, scoring and statistics are built and validated — this is the single remaining step.
2	A specification-coverage metric	Makes H2 testable at all.
3	A fifth condition: B with the sandbox	Separates role separation from grounded review.

Where the project stands

Twelve weeks, four phases, 445 automated tests across 14 modules, green on Python 3.11, 3.12 and 3.13. The engine runs, the benchmark is validated against its own reference solutions and a 91.4% mutation score, and the comparison apparatus is proven to detect a difference it was built to detect.

What is missing is not code. It is one API key and about an hour of compute.